

GRAND CHALLENGE MONOGRAPH COLLECTION

TYPE THEORY

The Grand Unified Theory of Computation

PROOF / PROGRAM

Logic Becomes Executable

Volume III

Grand Challenge Labs

Scope and series thesis

The phrase “grand unified theory” names a research thesis, not an established theorem. This volume asks why logical construction and computational construction repeatedly share the same laws.

The answer is deliberately stratified. In the constructive core CH-0, natural-deduction rules and typed term constructors admit an exact rule-level correspondence, and principal proof detours align with computational redexes. Dependent quantifiers are treated separately in CHD-1; sequent cut in LJ-0; classical/control interpretations in CHC-1; and parametricity only as the restricted preview CHF-1. Results are not transferred silently between these layers.

Volumes I and II are inherited at their actual institutional state: composition complete and durably admitted, with independent mathematical review pending. This volume does not upgrade that status.

Preface: why proof/program needs its own volume

A slogan can hide the work that makes it true. “Propositions are types” and “proofs are programs” become mathematically informative only after a logic, a term calculus, translations, reduction relations, and metatheorems have been fixed. The volume therefore starts small, proves exact correspondences where possible, and then subjects them to pressure: products, sums, dependent witnesses, cut, classical principles, erasure, extraction, polymorphism, proof-assistant trust, and finally open interaction.

The governing rule is simple: an analogy may motivate a construction, but it may not impersonate a theorem. Where the proof/program relation is exact, the text says why. Where a translation is required, it names the translation. Where evidence is only executable or finite, it remains bounded to that evidence.

Series orientation

Volume I asked what must exist before a program can be meaningful. Volume II asked how dependent formation principles build new computational worlds. Volume III asks why the laws of logical construction so often have computational form. Its endpoint is equally important: closed proof-term evaluation stops being an adequate primary model when computation becomes an open conversation. That threshold forces Volume IV, *PROTOCOL: Computation as Communication*.

Contents

Scope and series thesis	ii
Preface	iii
Series orientation	iv
1 A Proof That Runs	1
1.1 The smallest useful core	1
1.2 Term assignment rather than metaphysical identity	2
1.3 The composition proof is function composition	3
1.4 A first structural theorem: weakening	5
1.5 The checker as an executable microscope	5
1.6 What has actually been established	6
Exercises	6
2 Detours That Compute	8
2.1 Substitution is the hinge	8
2.2 Preservation is the type-level conservation law	9
2.3 A principal normalization correspondence	10
2.4 Normal forms and the strong theorem	12
2.5 The normalizer as executable evidence	12
2.6 Reduction is not definitional equality	13
2.7 What has actually been established	13
Exercises	14

3	Conjunction Carries Both	16
	Exercises	17
4	Disjunction Makes a Choice	20
	Exercises	22
5	Truth, Falsity, and Impossible Branches	24
	5.1 What strong normalization adds	26
	Exercises	27
6	Quantifiers Are Dependent Construction	29
	Exercises	30
7	Cut Is Substitution Wearing Different Clothes	33
	Exercises	35
8	What Constructive Logic Refuses to Guess	38
	Exercises	40
9	Classical Logic as Control	42
	Exercises	44
10	When Proofs Matter	46
	Exercises	48
11	Extraction	49
	Exercises	50
12	Uniformity Has Teeth	52
	Exercises	54
13	Proof Assistants Have a Trust Boundary	56
	Exercises	57
14	Where Proof / Program Stops Being Enough	59
	14.1 What survives the volume	60

Exercises	60
Formal reference and notation guide	63
Course and self-study roadmap	64
Illustration and production note	65
Selected bibliography	66
Transition to Volume IV	68

Chapter 1

A Proof That Runs

A proof and a program can look unrelated when they are written in their usual surface languages. A proof may be a tree of assumptions and inference rules. A program may be a term with variables, binders, and applications. The first purpose of this volume is to remove the accidental difference without erasing the real one.

Consider the elementary logical task

$$P \rightarrow Q, \quad Q \rightarrow R, \quad P \vdash R.$$

A natural-deduction proof applies the first implication to the evidence for P , obtaining evidence for Q , and then applies the second implication. A typed program does exactly the same thing: if $f : P \rightarrow Q$, $g : Q \rightarrow R$, and $x : P$, then $g(fx) : R$.

This is not yet a theorem about every logic and every programming language. It is an exact statement about the rules we now choose.

Governing Principle

The Curry–Howard correspondence begins only after the logical calculus and the term calculus have been fixed. The slogan comes after the rule map, not before it.

1.1 The smallest useful core

The primary calculus of this volume is CH-0. Its first tranche uses only implication. Types are generated by

$$A, B ::= P \mid A \rightarrow B,$$

where P ranges over atomic types. Terms are

$$t, u ::= x \mid \lambda x:A. t \mid tu.$$

A context Γ is a finite list of variable declarations $x:A$. The term judgment

$$\Gamma \vdash t : A$$

means that t has type A under assumptions Γ .

For the logical presentation we also write $\Gamma \vdash A$ when the proof term is being suppressed. The suppression is not proof irrelevance; it merely hides the selected witness from the notation.

Formal Interlude: implication rules

The implication fragment has the following rules.

$$\frac{}{\Gamma, x:A \vdash x : A} \text{VAR} \quad \frac{\Gamma, x:A \vdash t : B}{\Gamma \vdash \lambda x:A. t : A \rightarrow B} \rightarrow I$$

$$\frac{\Gamma \vdash f : A \rightarrow B \quad \Gamma \vdash a : A}{\Gamma \vdash fa : B} \rightarrow E.$$

The logical natural-deduction rules have the same introduction/elimination shape when the proof term is omitted. Assumption discharge in $\rightarrow I$ is represented by lexical binding in $\lambda x:A.t$; implication elimination is represented by application.

Plate 1.1 places these two presentations side by side. The point is not visual similarity. The point is that the premises and conclusion of each logical rule determine the constructor or eliminator of the assigned proof term.

1.2 Term assignment rather than metaphysical identity

Let D be a natural-deduction derivation in the implication fragment. We write $\llbracket D \rrbracket$ for its assigned proof term. The assignment is defined recursively:

- an open assumption labeled $x:A$ is assigned the variable x ;
- implication introduction discharging $x:A$ is assigned $\lambda x:A. \llbracket D \rrbracket$;
- implication elimination of proofs assigned f and a is assigned fa .

Proposition 1.1 (Rule-level term assignment). *If D is a well-formed implication-only natural-deduction derivation of A from assumptions represented by Γ , then the recursively assigned term satisfies*

$$\Gamma \vdash \llbracket D \rrbracket : A.$$

Proof. Induct on the final rule of D . If the final rule is an assumption, the result is exactly VAR. If it is implication introduction, the induction hypothesis gives the typing judgment for the immediate subderivation under the discharged assumption; rule $\rightarrow I$ produces the required abstraction. If it is implication elimination, the two induction hypotheses type the function evidence and argument evidence; rule $\rightarrow E$ types their application. These are all cases of the implication fragment. $\square$

The converse reconstruction is equally direct for this syntax: erase the term constructors into the corresponding introduction/elimination rules while preserving variable labels and types. There can still be administrative differences in how a human writes a derivation, so the volume speaks of a correspondence under a fixed representation rather than claiming that ink on a proof page is literally executable machine code.

Do not confuse

Do not confuse propositions-as-types with the claim that every type is merely a truth value. In CH-0, the same type-forming structure supports a logical reading and a computational reading. Data-bearing types may preserve distinctions that a proof-irrelevant semantics would quotient away. Later chapters make that pressure explicit.

1.3 The composition proof is function composition

The formula

$$(Q \rightarrow R) \rightarrow (P \rightarrow Q) \rightarrow (P \rightarrow R)$$

has the proof term

$$\lambda f:(Q \rightarrow R).\lambda g:(P \rightarrow Q).\lambda x:P.f(gx).$$

Read from the outside inward, the term says exactly what the proof says. Assume a proof/program f taking Q evidence to R evidence. Assume one taking P evidence to Q evidence. Assume P . Apply g , then f . Discharge the assumptions in reverse order.

Worked Example: composition as a proof object

Let

$$c = \lambda f:(Q \rightarrow R).\lambda g:(P \rightarrow Q).\lambda x:P.f(gx).$$

We can infer its type from the innermost application outward:

$$x : P, g : P \rightarrow Q \vdash gx : Q,$$

$$x : P, g : P \rightarrow Q, f : Q \rightarrow R \vdash f(gx) : R.$$

Three implication introductions then discharge x , g , and f . Thus

$$\vdash c : (Q \rightarrow R) \rightarrow (P \rightarrow Q) \rightarrow (P \rightarrow R).$$

No separate executable artifact has been smuggled in. The proof term itself is the typed composition program in this calculus.

ONE CONSTRUCTIVE OBJECT, TWO PRESENTATIONS

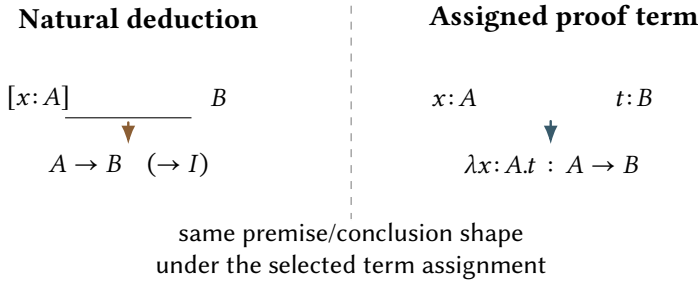


Figure 1.1: **Plate 1. One constructive object, two presentations.** Implication introduction discharges an assumption on the proof side and binds a variable on the term side. The alignment is exact for the selected CH-0 term assignment; it is not a claim that every proof formalism has this syntax.

THE IMPLICATION MACHINE

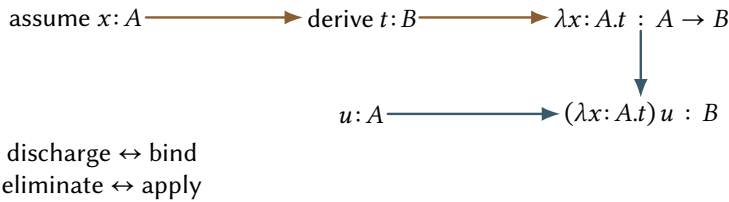


Figure 1.2: **Plate 3. The implication machine.** Assumption discharge becomes lexical binding, and modus ponens becomes application. The runtime reading does not replace semantic truth conditions; it exposes the constructive term carried by this calculus.

1.4 A first structural theorem: weakening

A proof can ignore an available assumption. A program can have an unused variable in its typing environment. These are the same structural permission at the level of CH-0 typing.

Theorem 1.2 (Weakening for the implication fragment). *If $\Gamma \vdash t : A$ and $y \notin \text{dom}(\Gamma)$, then*

$$\Gamma, y:B \vdash t : A.$$

Proof. Induct on the typing derivation of $\Gamma \vdash t : A$.

For VAR, the variable typed in Γ remains available in the extended context. For $\rightarrow I$, alpha-rename the bound variable if necessary so that it differs from y . Apply the induction hypothesis to the premise under the extended context, then reapply $\rightarrow I$. For $\rightarrow E$, apply the induction hypothesis to both premises and reapply application typing. No other typing rules occur in the implication fragment. $\square$

This proof matters because later substitution and preservation arguments depend on the discipline of contexts. The theorem is elementary, but its shape repeats throughout type theory: structural properties are not background magic; they are the infrastructure that makes local proof transformations legal.

1.5 The checker as an executable microscope

The first laboratory implements exactly the implication fragment just defined. Its surface syntax is intentionally austere:

```
Type: (-> P Q)
Term: (lam x P x)
Term: (app f a)
```

The checker parses these S-expressions, reconstructs types, and rejects mismatched applications or unbound variables. It is small enough that the reader can audit the entire trusted algorithm.

Computational Laboratory

Run `labs/lab01_implication_checker.py`. The retained fixture set checks identity, the K combinator, composition, a wrong argument type, and an unbound variable. The laboratory establishes only that this implementation behaves as recorded on these fixtures. The typing theorems remain mathematical obligations.

A successful run produces machine-readable JSON. The negative fixtures are as important as the positive ones: an implementation that accepts every term is not evidence for a type system.

1.6 What has actually been established

At this point the exact result is narrow and strong:

- CH-0 implication proofs admit a recursive term assignment.
- The assigned terms are well typed by Proposition 1.1.
- Assumption discharge and abstraction are aligned by rule shape.
- Implication elimination and application are aligned by rule shape.
- Weakening holds for the implication fragment.
- A small checker executes the same typing rules on retained examples.

We have not yet shown that proof simplification is computation. That requires a dynamic statement: when a proof contains an introduction immediately consumed by an elimination, what happens to the proof term? Chapter 2 answers that question.

Exercises

- 1.1. Checkpoint.** Give the CH-0 type of $\lambda x:P.x$.
- 1.2. Checkpoint.** Which typing rule corresponds to implication elimination? State both premises.
- 1.3. Checkpoint.** Explain why $\Gamma \vdash A$ suppresses information relative to $\Gamma \vdash t : A$ without assuming proof irrelevance.
- 1.4. Core.** Derive $P \rightarrow (Q \rightarrow P)$ and write its proof term.
- 1.5. Core.** Derive $(P \rightarrow Q) \rightarrow ((Q \rightarrow R) \rightarrow (P \rightarrow R))$ and compare the term with ordinary function composition.
- 1.6. Core.** In context $f : P \rightarrow Q, g : Q \rightarrow R, x : P$, type every proper subterm of $g(fx)$.
- 1.7. Synthesis.** Give one precise sense in which implication introduction and lambda abstraction are “the same structure,” and one sense in which the written proof and written program remain different artifacts.
- 1.8. Synthesis.** Why is “every type is a proposition” stronger than the correspondence established in this chapter?

- 1.9. **Proof Workshop.** Prove weakening for the implication fragment, explicitly handling the binder-renaming condition in the $\rightarrow I$ case.
- 1.10. **Proof Workshop.** Prove Proposition 1.1 by induction on the final natural-deduction rule and state why the induction is exhaustive.
- 1.11. **Design Clinic.** Extend the laboratory syntax with a human-readable arrow type parser while preserving the simple internal AST. State two malformed inputs that must still be rejected.
- 1.12. **Challenge.** Define a size measure on implication derivations and on assigned proof terms. Investigate which choices make rule-by-rule term assignment size-preserving and which introduce constant administrative overhead. State your metric before comparing sizes.

Chapter 2

Detours That Compute

A correspondence between static rules is suggestive but incomplete. Computation is about change. Proof normalization is also about change. The central test is therefore dynamic: does a local proof simplification induce the expected local program reduction?

Take a derivation that first proves $A \rightarrow B$ by assuming A , deriving B , and discharging the assumption; then immediately eliminates that implication using a proof of A . In natural deduction, the introduction followed immediately by elimination is a *detour*. The proof has constructed an implication only to consume it at once.

Under the term assignment, the same pattern is

$$(\lambda x:A.t)u.$$

Its beta contraction is

$$(\lambda x:A.t)u \longrightarrow t[u/x].$$

The dynamic core of Curry–Howard is now visible: principal proof normalization and beta computation move in lockstep under the chosen assignment.

2.1 Substitution is the hinge

The reduct $t[u/x]$ replaces the discharged assumption by the proof that supplied it. On the program side, it replaces a bound variable by an argument. Both readings require capture-avoiding substitution.

Formal Interlude: capture-avoiding substitution

For variables,

$$x[u/x] = u, \quad y[u/x] = y \quad (y \neq x).$$

For application,

$$(t_1 t_2)[u/x] = t_1[u/x] t_2[u/x].$$

For abstraction,

$$(\lambda y: B.t)[u/x]$$

leaves the body untouched if $y = x$; otherwise it substitutes recursively, alpha-renaming y first when y occurs free in u .

The alpha-renaming clause is not clerical trivia. Without it, substitution can change which assumption a variable denotes. The laboratory includes a retained capture-avoidance fixture for exactly this reason.

Lemma 2.1 (Substitution for the implication fragment). *If*

$$\Gamma, x: A \vdash t : B \quad \text{and} \quad \Gamma \vdash u : A,$$

then

$$\Gamma \vdash t[u/x] : B.$$

Proof. Induct on the derivation of $\Gamma, x: A \vdash t : B$.

If the final term is a variable, either it is x , in which case the conclusion is exactly the second premise, or it is another variable whose declaration remains in Γ .

If the final rule is application, apply the induction hypothesis to the function and argument premises and reapply $\rightarrow E$.

If the final rule is abstraction $\lambda y: C.s$, first alpha-rename y so that $y \neq x$ and y is fresh for the free variables of u . The premise has the form

$$\Gamma, x: A, y: C \vdash s : D.$$

By weakening, $\Gamma \vdash u : A$ remains valid under $y: C$. Apply the induction hypothesis to obtain

$$\Gamma, y: C \vdash s[u/x] : D,$$

then reapply $\rightarrow I$. □

2.2 Preservation is the type-level conservation law

A reduction step should simplify the term without changing the proposition/type it inhabits.

Theorem 2.2 (Preservation for principal implication beta reduction). *If*

$$\Gamma \vdash (\lambda x: A.t) u : B$$

and

$$(\lambda x: A.t) u \longrightarrow t[u/x],$$

then

$$\Gamma \vdash t[u/x] : B.$$

Proof. Invert the typing of the application. We obtain

$$\Gamma \vdash \lambda x:A.t : A \rightarrow B \quad \text{and} \quad \Gamma \vdash u : A.$$

Invert the typing of the abstraction to obtain

$$\Gamma, x:A \vdash t : B.$$

Lemma 2.1 then gives

$$\Gamma \vdash t[u/x] : B.$$

□

This proof is small because the substitution lemma absorbed the difficult bookkeeping. That pattern recurs across type systems: prove the structural substitution theorem once, then use it as the engine of subject reduction.

2.3 A principal normalization correspondence

We can now state the exact dynamic correspondence for the implication fragment.

Proposition 2.3 (Principal detour/beta correspondence). *Let a natural-deduction derivation end by applying implication elimination to an implication that was produced immediately by implication introduction from subderivation D with discharged assumption $x:A$, and let E derive A . Under the term assignment, the detour is assigned*

$$(\lambda x:A. \llbracket D \rrbracket) \llbracket E \rrbracket.$$

Contracting the principal logical detour corresponds to the beta step

$$(\lambda x:A. \llbracket D \rrbracket) \llbracket E \rrbracket \longrightarrow \llbracket D \rrbracket [\llbracket E \rrbracket / x].$$

The reduct has the same conclusion type as the original derivation.

Proof. The first statement follows directly from the recursive term assignment for $\rightarrow I$ and $\rightarrow E$. The proof-normalization step replaces every open occurrence of the discharged assumption in D by the supplied derivation E . The assigned term therefore performs capture-avoiding substitution of $\llbracket E \rrbracket$ for x in $\llbracket D \rrbracket$. Type preservation follows from Theorem 2.2. □

This is a genuine structural theorem. It is also deliberately local. Full natural-deduction normalization must handle all connectives and permutations/commuting conversions appropriate to the calculus. Full strong normalization must show that no well-typed reduction sequence can continue forever. Those are stronger claims.

INTRODUCTION THEN ELIMINATION IS A DETOUR

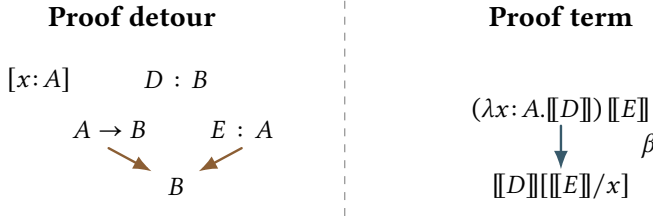
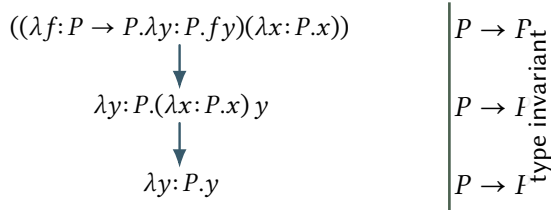


Figure 2.1: **Plate 2. Introduction then elimination is a detour.** The principal implication detour is assigned a beta redex, and proof substitution becomes capture-avoiding term substitution. This plate shows the principal implication case only; it does not prove global normalization.

A NORMALIZATION TRACE



trace generated by `lab02_normalizer.py`

Figure 2.2: **Plate 4. A normalization trace.** The teaching normalizer contracts two beta redexes and rechecks the invariant type after each step. This computed trace is evidence for the implementation on the retained fixture; one trace is not a proof of strong normalization.

Do not confuse

Do not confuse a normalization trace with a normalization theorem. A program that reduces every example in a test suite may still diverge on an untested input. Strong normalization is a universal mathematical claim over all well-typed terms in the stated calculus.

2.4 Normal forms and the strong theorem

A term is in beta-normal form when it contains no beta redex. For the simply typed lambda calculus, the standard theorem is much stronger than termination of our teaching evaluator: every well-typed term is strongly normalizing. We will cite that standard result for the full CH-0 core and later give a structured reducibility-candidates proof sketch that names the induction measure and function-type case.

The proof idea is not “types are finite, therefore execution ends.” The arrow type itself admits arbitrarily large terms. Reducibility arguments instead define a semantic predicate on typed terms strong enough to imply normalization and closed under application/substitution. For $A \rightarrow B$, a term is reducible when applying it to every reducible A -term yields a reducible B -term. Typing induction then places every well-typed term in the corresponding reducibility set.

That theorem belongs to the mathematical metatheory. The laboratory below has a different job: make individual reductions inspectable and automatically recheck the type invariant.

2.5 The normalizer as executable evidence

The second laboratory imports the exact AST and type checker from the first. It implements:

- free-variable computation;
- fresh-name generation;
- alpha-renaming for capture avoidance;
- capture-avoiding substitution;
- deterministic normal-order one-step reduction;
- trace generation;
- type rechecking after every step.

Computational Laboratory

Run `labs/lab02_normalizer.py`. Three retained fixtures cover a one-step identity beta reduction, a nested reduction, and a capture-avoidance case in which the binder must be alpha-renamed. The report explicitly states that these traces test the implementation and invariant on fixtures; they do not prove strong normalization.

Worked Example: capture avoidance

Suppose $y : P$ is free in the surrounding context and consider

$$(\lambda x : P. \lambda y : Q. x) y.$$

A naive textual substitution would produce $\lambda y : Q. y$, incorrectly capturing the free $y : P$ and even changing its intended type. Capture-avoiding substitution first renames the inner binder:

$$(\lambda x : P. \lambda y : Q. x) y \longrightarrow \lambda y_1 : Q. y.$$

The free $y : P$ remains free, and the reduct has type $Q \rightarrow P$.

2.6 Reduction is not definitional equality

The directed relation $t \longrightarrow u$ records a computation step. Definitional equality $t \equiv u$, when used, is an equational relation generated by selected computation laws and congruence; it can be symmetric even when evaluation is not. This distinction will matter more in dependent type theory and proof assistants, where conversion checking may use normalization or other decision procedures without identifying every equality proof with an evaluation step.

Similarly, proof normalization is a transformation on derivations. The term assignment lets us compare it with beta reduction, but the administrative syntax of derivations and terms can differ. Later, the sequent calculus will add a third representation: cut elimination. The volume will connect these three notions without merging their names.

2.7 What has actually been established

For the implication tranche we now have:

- a precise natural-deduction/term assignment;
- weakening;

- capture-avoiding substitution;
- preservation for principal beta reduction;
- a theorem identifying the principal implication detour with beta substitution under the assignment;
- executable traces that recheck the type invariant;
- a clearly separated cited obligation for strong normalization.

The next conceptual expansion is not to claim victory for “proofs are programs.” It is to test the correspondence against richer logical constructors. Conjunction must carry both pieces of evidence; disjunction must carry a choice. Those facts make proof relevance impossible to ignore.

Exercises

- 2.1. Checkpoint.** Contract $(\lambda x:P.x)u$ by one beta step.
- 2.2. Checkpoint.** State the two premises of the substitution lemma.
- 2.3. Checkpoint.** Why must substitution alpha-rewrite a binder that occurs free in the replacement term?
- 2.4. Core.** Normalize $(\lambda f:(P \rightarrow P).\lambda x:P.fx)(\lambda z:P.z)$ and annotate the type after every step.
- 2.5. Core.** Give an example in which naive textual substitution captures a variable. Then perform the capture-avoiding substitution correctly.
- 2.6. Core.** Reconstruct the inversion steps used in the proof of Theorem 2.2.
- 2.7. Synthesis.** Explain the relation among proof detour elimination, beta reduction, and substitution without using the words “they are identical.”
- 2.8. Synthesis.** Give a concrete reason that deterministic normal-order evaluation and the mathematical one-step beta relation are different objects.
- 2.9. Proof Workshop.** Complete the abstraction case of Lemma 2.1, including the weakening step for the substituted argument.
- 2.10. Proof Workshop.** Prove preservation for a beta step occurring strictly inside the function position of an application, assuming compatible closure of reduction and the induction hypothesis for the subterm.
- 2.11. Design Clinic.** Modify the normalizer design so that every trace step includes the redex path contracted. Specify a stable path representation and one invariant the trace checker should enforce.

- 2.12. Challenge.** Write a precise statement of strong normalization for CH-0. Then explain why termination of the provided normal-order evaluator is logically weaker than that statement, even if the evaluator terminates on every fixture in this volume.

Chapter 3

Conjunction Carries Both

Implication showed how a discharged assumption becomes a function. Conjunction adds a different demand: constructive evidence for $A \wedge B$ must provide both components. In CH-0 the corresponding type is $A \times B$, and its canonical constructor is a pair.

Product rules

$$\frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B}{\Gamma \vdash \langle a, b \rangle : A \times B} \times I$$
$$\frac{\Gamma \vdash p : A \times B}{\Gamma \vdash \pi_1 p : A} \times E_1 \quad \frac{\Gamma \vdash p : A \times B}{\Gamma \vdash \pi_2 p : B} \times E_2.$$

The principal reductions are $\pi_1 \langle a, b \rangle \longrightarrow a$ and $\pi_2 \langle a, b \rangle \longrightarrow b$.

The logical reading is exact at the rule level: conjunction introduction requires both derivations; each elimination selects one component. The computational reading is equally direct: construct a pair; project one field.

Proposition 3.1 (Product term assignment). *If a natural-deduction derivation in the $\{\rightarrow, \times\}$ fragment is assigned terms recursively by mapping conjunction introduction to pairing and conjunction elimination to projection, then the assigned term has the derived formula as its CH-0 type.*

Proof. Extend the induction of Chapter 1. The only new final-rule cases are $\times I$, $\times E_1$, and $\times E_2$; each is exactly the corresponding typing rule displayed above. The implication cases are unchanged. $\square$

Lemma 3.2 (Product canonical form). *If v is a closed CH-0 value and $\vdash v : A \times B$, then $v = \langle v_1, v_2 \rangle$ for values v_1, v_2 with $\vdash v_1 : A$ and $\vdash v_2 : B$.*

Proof. Inspect the value grammar and invert the final typing rule. A lambda has arrow type, an injection has sum type, and $\star$ has unit type. The only value constructor whose conclusion has product type is pairing. Inversion of $\times I$ gives the component types. $\square$

A proof object that cannot forget either side

From $f : P \rightarrow Q$ and $x : P$, the term

$$\langle x, fx \rangle : P \times Q$$

contains evidence for both claims. Projecting the first component returns x ; projecting the second computes fx . A proof-irrelevant truth value for “ P and Q ” would not by itself expose these two computationally distinct projections.

Theorem 3.3 (Principal product preservation). *If $\Gamma \vdash \pi_i \langle a, b \rangle : C$ and the principal product redex contracts to t , then $\Gamma \vdash t : C$.*

Proof. For $i = 1$, inversion yields $\Gamma \vdash a : A$ and $C = A$; the reduct is a . The $i = 2$ case is symmetric. $\square$

Do not confuse

Conjunction/product correspondence does not mean that every semantic product is a proposition or that all proofs of a conjunction are equal. The chapter establishes a constructor/eliminator correspondence in CH-0.

Computational Laboratory

Run `python labs/lab03_products.py`. The laboratory enumerates retained well-typed product fixtures, checks both projections, and rejects malformed projection heads. Its JSON report is implementation evidence only.

Exercises

- 3.1. **Checkpoint.** State the three product typing rules.
- 3.2. **Checkpoint.** Contract $\pi_2 \langle a, b \rangle$.
- 3.3. **Checkpoint.** What extra information does a pair contain beyond the assertion that both components are inhabited?
- 3.4. **Core.** Type $\lambda x : P. \langle x, x \rangle$.
- 3.5. **Core.** Derive $(P \times Q) \rightarrow (Q \times P)$ and give its term.
- 3.6. **Core.** Normalize $\pi_1 \langle (\lambda x : P. x)u, v \rangle$ under compatible closure.
- 3.7. **Synthesis.** Compare conjunction elimination with record projection.
- 3.8. **Synthesis.** Explain why product canonical forms are a typing theorem, not a statement about arbitrary runtime data.

PRODUCT DETOURS

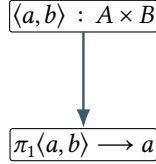


Figure 3.1: **Plate 7. Product detours.** Pair construction immediately followed by projection contracts to the selected component. Scope: principal CH-0 product reduction only.

PRODUCT CANONICAL FORM

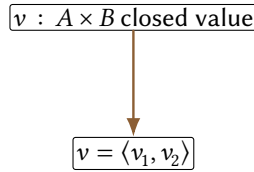


Figure 3.2: **Plate 8. Product canonical form.** A closed value of product type must be a pair under the CH-0 value grammar. Scope: a typing inversion fact, not a claim about arbitrary runtime objects.

BOTH PIECES SURVIVE

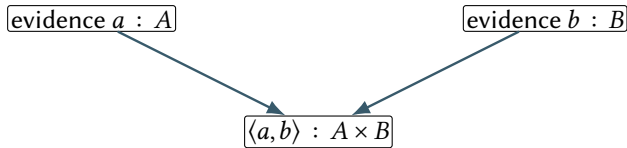


Figure 3.3: **Plate 9. Both pieces survive.** Constructive conjunction packages both witnesses, making projection computationally meaningful. Scope: no global proof-irrelevance conclusion is drawn.

- 3.9. Proof Workshop.** Extend substitution through the pair and projection cases.
- 3.10. Proof Workshop.** Prove both principal product-preservation cases.
- 3.11. Design Clinic.** Specify negative fixtures a product checker must reject.
- 3.12. Challenge.** Formulate product eta as an equational law and explain why it is not a one-step operational reduction in this volume.

Chapter 4

Disjunction Makes a Choice

A constructive proof of $A \vee B$ does more than certify that one side holds. It exposes which side was constructed and carries the evidence needed by that branch. In CH-0, this is a sum type.

Sum rules

$$\frac{\Gamma \vdash a : A}{\Gamma \vdash \text{inl}(a) : A + B} + I_1 \quad \frac{\Gamma \vdash b : B}{\Gamma \vdash \text{inr}(b) : A + B} + I_2$$
$$\frac{\Gamma \vdash s : A + B \quad \Gamma, x:A \vdash t : C \quad \Gamma, y:B \vdash u : C}{\Gamma \vdash \text{case } s \text{ of } \text{inl}(x) \Rightarrow t \mid \text{inr}(y) \Rightarrow u : C} + E.$$

Case analysis is typed branching: the scrutinee chooses a branch, but both branches must converge on the same result type C .

Lemma 4.1 (Sum canonical form). *If v is closed, $\vdash v : A + B$, and v is a value, then either $v = \text{inl}(v_A)$ with $\vdash v_A : A$, or $v = \text{inr}(v_B)$ with $\vdash v_B : B$.*

Proof. By inspection of the value grammar and inversion of typing. No other value constructor concludes with a sum type. $\square$

Theorem 4.2 (Principal case preservation). *A well-typed case expression whose scrutinee is a matching injection contracts to a well-typed selected branch of the same result type.*

Proof. For the left case, inversion gives $\Gamma \vdash a : A$ and $\Gamma, x:A \vdash t : C$. The reduct is $t[a/x]$, typed C by substitution. The right case is symmetric. $\square$

Choice is evidence

The term

$$\lambda x:P. \text{inl}(x) : P \rightarrow (P + Q)$$

DISJUNCTION MAKES A CHOICE

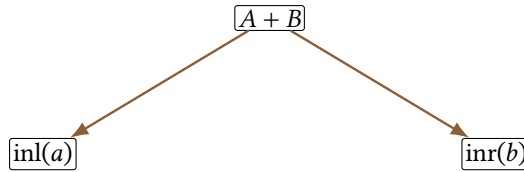


Figure 4.1: **Plate 10. Disjunction makes a choice.** A constructive sum inhabitant records a branch and its payload. Scope: richer than a proof-irrelevant Boolean, but not a universal semantics of disjunction.

CASE BRANCHES CONVERGE

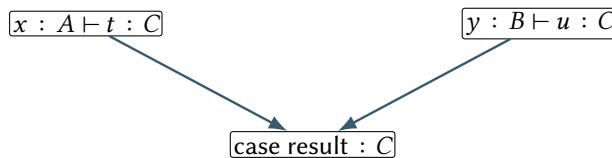


Figure 4.2: **Plate 11. Case branches converge.** Sum elimination may inspect either constructor, but both branches must produce the same result type. Scope: the stated CH-0 rule.

not only proves the disjunction; it records that the left alternative was chosen. A consumer can inspect that constructor and run the corresponding branch. This computational choice is precisely what a constructive disjunction proof contributes.

Do not confuse

A sum inhabitant is not merely a Boolean bit. It carries a tag and a payload. Conversely, this chapter does not assert that every logical disjunction in every semantics should be implemented as a tagged union.

Computational Laboratory

Run `python labs/lab04_sums.py`. It evaluates retained left/right case fixtures, verifies branch-result type agreement, and rejects mismatched branch codomains.

Exercises

- 4.1. **Checkpoint.** State both sum introduction rules.
- 4.2. **Checkpoint.** Why must both case branches have type C ?
- 4.3. **Checkpoint.** Contract a case over $\text{inr}(b)$.
- 4.4. **Core.** Derive $P \rightarrow (P + Q)$.
- 4.5. **Core.** Derive $(P + Q) \rightarrow (Q + P)$.
- 4.6. **Core.** Type a case expression that maps $P + Q$ into a common type R given $P \rightarrow R$ and $Q \rightarrow R$.
- 4.7. **Synthesis.** Contrast sum evidence with a Boolean truth flag.
- 4.8. **Synthesis.** Explain how case analysis exposes proof relevance.
- 4.9. **Proof Workshop.** Prove substitution for the case constructor, including both binders.
- 4.10. **Proof Workshop.** Prove principal case preservation for the right injection.
- 4.11. **Design Clinic.** Specify an AST and type-checking rule for case expressions.
- 4.12. **Challenge.** State a sum eta principle and classify it as operational, equational, or semantic under the conventions of this volume.

SUM DETOUR

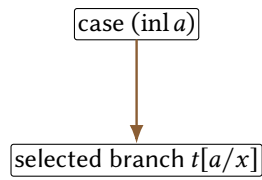


Figure 4.3: **Plate 12. Sum detour.** Injection followed immediately by case analysis contracts to the selected branch by substitution. Scope: principal reduction only; no arbitrary-extension confluence claim.

Chapter 5

Truth, Falsity, and Impossible Branches

The unit and empty types sharpen the distinction between construction and assertion. The unit type has a canonical constructor $\star$. The empty type has none. Empty elimination is powerful precisely because its premise cannot be constructed in a consistent closed CH-0 computation.

Unit and empty rules

$$\frac{}{\Gamma \vdash \star : \mathbf{1}} \text{1I} \quad \frac{\Gamma \vdash e : \mathbf{0}}{\Gamma \vdash \text{abort}_A(e) : A} \text{0E}.$$

There is no introduction rule for $\mathbf{0}$.

Lemma 5.1 (Canonical forms for CH-0). *Let v be a closed value.*

1. *If $\vdash v : A \rightarrow B$, then v is a lambda abstraction.*
2. *If $\vdash v : A \times B$, then v is a pair.*
3. *If $\vdash v : A + B$, then v is a left or right injection.*
4. *If $\vdash v : \mathbf{1}$, then $v = \star$.*
5. *There is no closed value v with $\vdash v : \mathbf{0}$.*

Proof. Each clause follows by inspecting the value constructors and inverting the typing rule that could assign the stated outer type. The final clause follows because the value grammar contains no empty constructor and no other constructor can be typed $\mathbf{0}$ by inversion. $\square$

Theorem 5.2 (Progress for closed CH-0). *If $\vdash t : A$, then either t is a value or there exists t' with $t \longrightarrow t'$ under the deterministic teaching evaluator.*

TRUTH HAS A CONSTRUCTOR

$$\boxed{\star : \mathbf{1}}$$

Figure 5.1: **Plate 13. Truth has a constructor.** The unit type has the canonical value $\star$, giving the constructive reading of truth. Scope: this does not imply equality of arbitrary proof objects at other types.

EMPTY HAS NO CONSTRUCTOR

$$\boxed{\text{no closed value } v : \mathbf{0}}$$

$$\downarrow$$

$$\boxed{e : \mathbf{0} \Rightarrow \text{abort}_A(e) : A}$$

Figure 5.2: **Plate 14. Empty has no constructor.** Empty elimination consumes an impossible premise; it does not fabricate a closed inhabitant of $\mathbf{0}$. Scope: the CH-0 canonical-form argument.

Proof. Induct on the typing derivation. Variables cannot occur in a closed derivation. Introduction forms are values once their recursively designated value premises are values. For elimination forms, apply the induction hypothesis to the principal subterm. If it steps, the whole term steps by the evaluator context. If it is a value, canonical forms expose the matching constructor: lambdas for application, pairs for projection, injections for case analysis. For $\text{abort}_A(e)$, the induction hypothesis says e either steps or is a value; the latter is impossible by the empty canonical-form clause. Hence every nonvalue closed well-typed term has a next evaluator step. $\square$

Theorem 5.3 (Preservation for CH-0). *If $\Gamma \vdash t : A$ and $t \longrightarrow t'$, then $\Gamma \vdash t' : A$.*

Proof. Induct on the derivation of the evaluator step. Principal beta uses substitution. Product projections use inversion. Sum cases use substitution in the selected branch. Congruence/evaluation-context cases reapply the surrounding typing rule to the induction hypothesis. Unit has no reduction, and empty elimination can only step through its premise. $\square$

5.1 What strong normalization adds

Progress and preservation say that well-typed closed computation does not become stuck. They do not forbid an infinite sequence of well-typed steps. For pure simply typed lambda calculus with products, sums, unit, and empty, strong normalization is standard. We use that theorem as an imported result and give its proof architecture rather than pretending the fixture suite proves it.

Define a reducibility predicate $\mathcal{R}_A(t)$ by induction on A : at atomic types it requires strong normalization; at arrows it additionally requires that tu be reducible at B for every reducible u at A ; products require reducible projections; sums require strong normalization together with reducible payloads when a normal form exposes an injection; unit is strongly normalizing; empty is strongly normalizing with no closed canonical inhabitant. The fundamental lemma proves that a well-typed term under a reducible substitution is reducible. The identity substitution then yields strong normalization for well-typed terms. This is a structured proof sketch, not an independent re-proof of every reducibility lemma.

Do not confuse

Strong normalization here is scoped to pure CH-0. It is not transferred to the later continuation layer, unrestricted dependent type theory, effects, recursion, or open protocols.

Computational Laboratory

Run `python labs/lab05_empty_unit.py`. The laboratory enumerates the retained canonical values, tests progress on a finite fixture suite, and verifies that the parser/checker exposes no empty constructor.

Exercises

- 5.1. **Checkpoint.** Give the canonical inhabitant of **1**.
- 5.2. **Checkpoint.** Why is there no introduction rule for **0**?
- 5.3. **Checkpoint.** State progress without confusing it with normalization.
- 5.4. **Core.** Type $\lambda e:0. \text{abort}_p(e)$.
- 5.5. **Core.** Classify the possible closed values at each CH-0 type former.
- 5.6. **Core.** Trace progress for an application whose function subterm is not yet a value.
- 5.7. **Synthesis.** Explain why `ex falso` does not manufacture a closed inhabitant of **0**.
- 5.8. **Synthesis.** Distinguish type safety, evaluator termination, and strong normalization.
- 5.9. **Proof Workshop.** Fill in the case-analysis branch of progress.
- 5.10. **Proof Workshop.** Fill in the congruence cases of preservation.
- 5.11. **Design Clinic.** Design a fixture generator that cannot accidentally assume the theorem it tests.
- 5.12. **Challenge.** State the fundamental lemma needed by the reducibility argument, including the reducible-substitution hypothesis.

CANONICAL FORMS BY TYPE

$A \rightarrow B$:	lambda
$A \times B$:	pair
$A + B$:	injection
1 :	★
0 :	none

Figure 5.3: **Plate 15. Canonical forms by type.** Outer type constrains the possible shape of a closed value and powers the progress proof. Scope: scoped to the declared CH-0 value grammar.

Chapter 6

Quantifiers Are Dependent Construction

The propositions-as-types reading becomes substantially more expressive when the type of the evidence may depend on a value. Volume II developed the dependent formation rules. Here we reuse them without changing their meaning and ask what logical construction they encode.

For a family $x : A \vdash B(x)$ type, a term of $\Pi(x : A).B(x)$ is a construction that, given each $a : A$, produces evidence of $B(a)$. A term of $\Sigma(x : A).B(x)$ is a witness $a : A$ paired with evidence of $B(a)$.

Dependent universal/existential reading

The core rules used here are

$$\frac{\Gamma, x : A \vdash b : B(x)}{\Gamma \vdash \lambda x. b : \Pi(x : A).B(x)} \quad \frac{\Gamma \vdash f : \Pi(x : A).B(x) \quad \Gamma \vdash a : A}{\Gamma \vdash fa : B(a)}$$

and

$$\frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B(a)}{\Gamma \vdash \langle a, b \rangle : \Sigma(x : A).B(x)}$$

Elimination for Σ exposes a witness together with its dependent evidence under the usual substitution discipline.

Proposition 6.1 (Rule-level quantifier correspondence). *Under the CHD-1 reading, dependent product introduction/elimination has the constructor/eliminator shape of constructive universal introduction/instantiation, and dependent sum introduction/elimination has the witness/evidence shape of constructive existential introduction/elimination.*

Proof. Read the displayed premises and conclusions propositionally. Π introduction abstracts over an arbitrary $x : A$ and returns evidence whose type is $B(x)$; elimination instantiates at a supplied a . Σ introduction supplies both the witness and evidence at that

witness. Its eliminator binds both pieces before producing the motive result. The correspondence is rule-by-rule; no normalization result beyond the separately cited dependent metatheory is used. $\square$

Finite universal construction

Let `Bit` have constructors `0`, `1`, and let $B(b)$ be a family whose evidence records that flipping twice returns b . A dependent function

$$f : \Pi(b : \text{Bit}).B(b)$$

contains a method that works for either input. By contrast, a dependent pair

$$p : \Sigma(b : \text{Bit}).B(b)$$

contains one selected bit and evidence about that bit. Universal construction supplies a procedure; existential construction supplies a witness.

Do not confuse

This chapter does not promote the full dependent metatheory. Volume II is inherited at its actual durable-but-unreviewed state, and CH-0 strong normalization is not silently generalized to all of CHD-1.

Computational Laboratory

Run `python labs/lab06_quantifiers.py`. The finite-domain laboratory checks explicit universal tables and existential witness/evidence packages. It is a bounded model of the rule shapes, not a dependent-type-theory kernel.

Exercises

- 6.1. Checkpoint.** Read $\Pi(x : A).B(x)$ constructively.
- 6.2. Checkpoint.** Read $\Sigma(x : A).B(x)$ constructively.
- 6.3. Checkpoint.** What substitution occurs when a dependent function is applied to $a : A$?
- 6.4. Core.** Write the type of a function returning a proof family over a two-element domain.
- 6.5. Core.** Construct a dependent pair and identify its witness and evidence components.
- 6.6. Core.** Explain why ordinary $A \rightarrow B$ is a special case of Π when B does not depend on x .

UNIVERSAL CONSTRUCTION

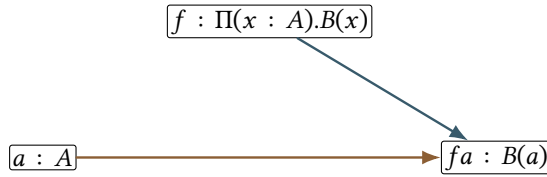


Figure 6.1: **Plate 16. Universal construction as dependent function.** A Π -term supplies evidence at each argument. Scope: the inherited CHD-1 rules, not every semantics of universal quantification.

EXISTENTIAL EVIDENCE

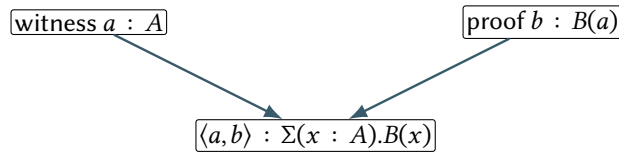


Figure 6.2: **Plate 17. Existential evidence is witness plus proof.** A dependent sum carries both a selected witness and evidence specialized to it. Scope: witness erasure requires a separate policy.

DEPENDENT SUBSTITUTION

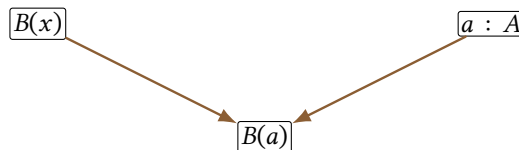


Figure 6.3: **Plate 18. Quantifier substitution.** Instantiation substitutes a witness into the dependent proposition/type family. Scope: capture avoidance and well-formed dependency remain formal obligations.

- 6.7. **Synthesis.** Compare existential evidence with a database record whose second field is indexed by the first.
- 6.8. **Synthesis.** Explain why witness erasure can change the computational interpretation of an existential proof.
- 6.9. **Proof Workshop.** Prove the rule-level Π correspondence by induction on the selected derivation forms.
- 6.10. **Proof Workshop.** State the substitution side condition needed by Σ elimination.
- 6.11. **Design Clinic.** Specify a finite-domain checker for dependent witness packages and one malformed package it must reject.
- 6.12. **Challenge.** Identify exactly which CH-0 metatheorems would require new proof obligations before being claimed for a richer dependent calculus.

Chapter 7

Cut Is Substitution Wearing Different Clothes

Natural deduction makes assumption discharge visible. Sequent calculus makes composition visible. The rule called *cut* inserts an intermediate formula and composes a proof of that formula with a proof that uses it. This is closely related to substitution, but the theorem statements live in different proof presentations.

We use a bounded single-succedent intuitionistic sequent calculus LJ-0. Sequents have the form $\Gamma \Rightarrow A$. Identity and cut are

$$\frac{}{\Gamma, A \Rightarrow A} \text{ id} \quad \frac{\Gamma \Rightarrow A \quad \Delta, A \Rightarrow C}{\Gamma, \Delta \Rightarrow C} \text{ cut}.$$

Logical left/right rules are given for $\rightarrow, \times, +, \mathbf{1}, \mathbf{0}$ in the standard intuitionistic style.

A cut can be read as explicit composition. Under a translation back to natural deduction, the right premise is a derivation with an open assumption A , and the left premise supplies what is substituted for that assumption. Under term assignment, the same movement becomes capture-avoiding substitution. These facts explain the family resemblance among cut elimination, proof normalization, and beta reduction without collapsing them into one syntactic operation.

Proposition 7.1 (Principal implication cut reduction). *In LJ-0, a cut whose cut formula $A \rightarrow B$ is introduced last on the right in the left premise and consumed last by the matching implication-left rule in the right premise can be transformed into cuts on structurally smaller formulas/premises.*

Proof. Expose the two final inference rules. Replace the cut on $A \rightarrow B$ by composition of the proof of the implication body with the proof of the argument assumption and then with the continuation deriving the final succedent. The resulting cuts have lower logical complexity or lower proof height under the standard lexicographic cut measure. This is the principal local reduction; commuting conversions handle nonprincipal occurrences. $\square$

TWO PROOF PRESENTATIONS



Figure 7.1: **Plate 19. Natural-deduction and sequent views.** Translations connect two proof presentations with different administrative structure. Scope: correspondence, not literal identity of derivations.

CUT AS COMPOSITION

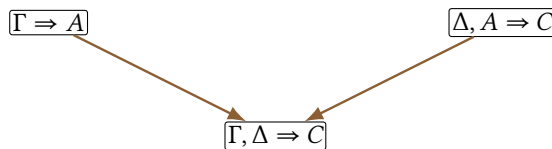


Figure 7.2: **Plate 20. Cut as explicit composition.** The cut formula is an explicit intermediate result consumed by the second derivation. Scope: LJ-0 only.

Imported theorem: cut admissibility

Gentzen's Hauptsatz provides the historical prototype for cut elimination in sequent calculi. For the standard intuitionistic propositional sequent calculus matching LJ-0, we use cut admissibility as a cited theorem: if a sequent is derivable with cut, it has a cut-free derivation. This volume demonstrates principal and bounded commuting reductions but does not claim a fresh independent proof of the full theorem.

Do not confuse

Normalization, beta reduction, and cut elimination are related through translations. They are not interchangeable labels. Their objects, reduction steps, and induction measures differ.

Computational Laboratory

Run `python labs/lab07_cut_reducer.py`. The bounded reducer constructs finite proof trees, contracts selected principal cuts, and checks that the end sequent is preserved while a declared cut measure decreases. It is not evidence for arbitrary cut elimination.

Exercises

- 7.1. **Checkpoint.** State the cut rule.
- 7.2. **Checkpoint.** What is the cut formula?
- 7.3. **Checkpoint.** Why can cut be read as explicit composition?
- 7.4. **Core.** Translate a simple implication application into a cut-containing sequent derivation.
- 7.5. **Core.** Identify the substitution corresponding to that cut.
- 7.6. **Core.** Give a lexicographic cut measure using formula complexity and proof height.
- 7.7. **Synthesis.** Compare natural-deduction detours with sequent cuts.
- 7.8. **Synthesis.** Explain why a bounded cut reducer cannot establish the Hauptsatz.
- 7.9. **Proof Workshop.** Work the principal implication cut reduction in detail.
- 7.10. **Proof Workshop.** Give one commuting conversion for a nonprincipal cut.
- 7.11. **Design Clinic.** Specify a proof-tree representation that records end sequents and cut formulas.

- 7.12. Challenge.** State precisely what must be proved for cut admissibility rather than merely local cut reduction.

CUT REDUCTION

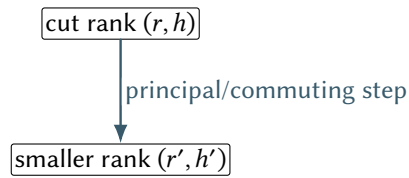


Figure 7.3: **Plate 21. Cut reduction trace.** A bounded cut step preserves the end sequent while decreasing the declared measure. Scope: computed trace is not the general cut-elimination theorem.

Chapter 8

What Constructive Logic Refuses to Guess

The constructive core has a simple discipline: to prove a disjunction, construct a branch; to prove an existential, supply a witness. Classical logic validates principles that need not expose such information. The difference is not a defect in either system; it is a difference in admissible proof principles.

Let $\neg A$ abbreviate $A \rightarrow \mathbf{0}$. The schema $A + \neg A$ is excluded middle in the propositional reading. It is not derivable from the pure CH-0 rules alone.

To make that nonderivability visible, consider the two-world Kripke frame $w_0 \leq w_1$. Let atomic P be false at w_0 and forced at w_1 . Persistence holds because once P becomes true it remains true upward.

At w_0 , P is not forced. Neither is $\neg P$: forcing $P \rightarrow \mathbf{0}$ at w_0 would require every extension forcing P also to force $\mathbf{0}$, but w_1 forces P and not $\mathbf{0}$. Therefore w_0 forces neither disjunct, so it does not force $P + \neg P$.

Proposition 8.1 (Finite countermodel to excluded middle). *The Kripke model above does not force $P \vee \neg P$ at its root.*

Proof. By the forcing clauses for disjunction and implication just evaluated: neither P nor $P \rightarrow \mathbf{0}$ is forced at w_0 . □

By the standard soundness theorem for intuitionistic propositional logic with respect to Kripke semantics, the existence of this countermodel implies that excluded middle is not derivable in the corresponding intuitionistic calculus.

Information not supplied

A classical proof may establish that one of P or $\neg P$ must hold without constructing which branch in the same direct manner as $+I_1$ or $+I_2$. If we want executable

A WORLD WHERE EXCLUDED MIDDLE FAILS

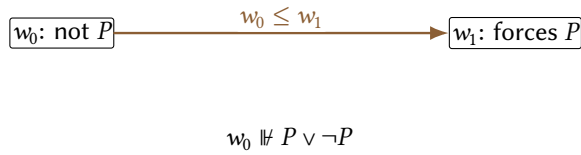


Figure 8.1: **Plate 22. Finite Kripke countermodel.** At the root, neither P nor $\neg P$ is forced, so excluded middle fails there. Scope: selected intuitionistic Kripke semantics.

CONSTRUCTIVE INFORMATION

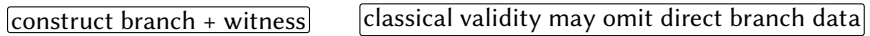


Figure 8.2: **Plate 23. Constructive information versus classical commitment.** Constructive introduction rules expose branch/witness data that classical validity need not provide in the same form. Scope: calculus-relative, not a value judgment.

content from such reasoning, we need a translation, additional semantic structure, or control behavior. Chapter 9 explores one such route.

Do not confuse

“Nonconstructive” is calculus-relative. It does not mean invalid, useless, or irrational. It says that the proof uses principles not justified by the witness-producing rules fixed for CH-0.

Computational Laboratory

Run `python labs/lab08_kripke.py`. The finite model checker verifies persistence and evaluates the selected excluded-middle formula at both worlds. Its result is a countermodel for that formula under the implemented forcing semantics, not a survey of all constructive logics.

Exercises

- 8.1. Checkpoint.** Define $\neg A$ in CH-0.
- 8.2. Checkpoint.** What evidence does constructive disjunction require?
- 8.3. Checkpoint.** Why does w_0 fail to force $\neg P$ in the example?
- 8.4. Core.** Evaluate $P \vee \neg P$ at w_1 .
- 8.5. Core.** State the persistence condition for atomic forcing.
- 8.6. Core.** Build a finite model separating another selected classical schema from intuitionistic validity.
- 8.7. Synthesis.** Explain how a semantic countermodel becomes a syntactic nonderivability result only with a soundness theorem.
- 8.8. Synthesis.** Contrast lack of a constructive proof with falsity in a classical two-valued model.
- 8.9. Proof Workshop.** Prove persistence for the implication forcing clause by induction on formulas.
- 8.10. Proof Workshop.** Verify every forcing clause used in the root countermodel.
- 8.11. Design Clinic.** Specify checks that a finite Kripke-model parser should impose before evaluation.
- 8.12. Challenge.** Explain why adding excluded middle changes proof theory even when ordinary classical truth tables remain familiar.

DOUBLE NEGATION IS A BOUNDARY



Figure 8.3: **Plate 24. Double negation as a translation boundary.** Classical reasoning can enter a constructive target through a negative translation rather than literal proof identity. Scope: selected translation only.

Chapter 9

Classical Logic as Control

Classical reasoning need not be forced into the pure constructive core. A more informative strategy is to translate it. Continuation-passing style makes the future use of a computation explicit as an argument. This gives certain classical principles a computational reading in terms of control.

Fix an answer type R . For the restricted arrow fragment used here, define a call-by-name CPS target by assigning each source computation $M : A$ a target term $M^{\text{cps}} : (A^* \rightarrow R) \rightarrow R$. Atomic types translate to themselves; arrows translate to functions between CPS-aware values according to the chapter's displayed grammar. Variables, abstraction, and application are translated structurally with fresh continuations.

Theorem 9.1 (Typing preservation for the restricted CPS translation). *For the source fragment explicitly defined in this chapter, if $\Gamma \vdash M : A$, then the translated context types M^{cps} at $(A^* \rightarrow R) \rightarrow R$.*

Proof. Induct on the source typing derivation. The variable case passes the translated value to the current continuation. The abstraction case builds a target function whose body satisfies the induction hypothesis under the translated argument binding. The application case sequences the translated function and argument through fresh continuations and then invokes the function value. Each continuation is assigned exactly the answer type R , so the target typing derivation closes. The theorem is only for the displayed translation; it is not a normalization theorem for arbitrary control operators. $\square$

Double negation provides the logical shape behind this translation: a CPS computation of result A can be read as expecting a continuation $A \rightarrow R$. With $R = \mathbf{0}$, the outer shape resembles $\neg\neg A$. Classical reasoning can therefore be embedded into a constructive target by a negative translation, while control operators can make continuation structure operationally explicit.

CONTINUATION-PASSING CONTROL

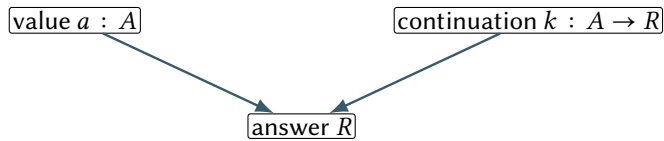


Figure 9.1: **Plate 25. Continuation-passing control flow.** CPS makes the future use of a value an explicit typed argument. Scope: the restricted CHC-1 translation, not pure CH-0.

CLASSICAL PROOF AS CONTROL



Figure 9.2: **Plate 26. Classical proof as control.** Selected classical principles receive computational readings through continuation structure. Scope: one typed interpretation among several, not universal identity.

The continuation is part of the term

A direct-style computation asks “what value do I return?” Its CPS translation asks “what should I do with the value next?” Making that second question explicit turns evaluation context into a typed parameter. Certain classical proof steps can then be interpreted as manipulating that context rather than conjuring a constructive witness from nowhere.

Do not confuse

This chapter establishes a typed translation and selected control examples. It does not state that classical logic and control operators are literally identical, nor that CHC-1 inherits CH-0 strong normalization or extraction properties.

Computational Laboratory

Run `python labs/lab09_cps.py`. The laboratory checks the translation on a finite typed arrow-term suite and compares direct and CPS results for closed fixtures. The agreement is fixture-scoped.

Exercises

- 9.1. **Checkpoint.** What is a continuation?
- 9.2. **Checkpoint.** Why is an answer type R fixed in the restricted translation?
- 9.3. **Checkpoint.** What logical shape appears when $R = 0$?
- 9.4. **Core.** CPS-translate a typed identity term under the displayed rules.
- 9.5. **Core.** Track the types of the continuations introduced by an application translation.
- 9.6. **Core.** Compare direct and CPS evaluation of a closed composition example.
- 9.7. **Synthesis.** Explain why translation is weaker than literal identity but stronger than a loose analogy.
- 9.8. **Synthesis.** Explain why control can threaten naive transfer of extraction claims.
- 9.9. **Proof Workshop.** Write the abstraction case of CPS typing preservation in full.
- 9.10. **Proof Workshop.** Write the application case in full, including fresh-variable assumptions.
- 9.11. **Design Clinic.** Specify an alpha-renaming discipline for a CPS translator.
- 9.12. **Challenge.** Identify which theorem would have to be re-proved before claiming strong normalization for an extended control calculus.

DIRECT STYLE / CPS



Figure 9.3: **Plate 27. Direct style versus CPS.** A direct result becomes a computation parameterized by its continuation. Scope: typing preservation is proved only for the displayed restricted translation.

Chapter 10

When Proofs Matter

Propositions-as-types does not entail proof irrelevance. In the calculi used so far, proof terms are ordinary typed terms. Two inhabitants can carry different data, select different branches, or induce different runtime behavior. Erasure is therefore an additional policy, not a consequence of the correspondence itself.

Call an argument *computationally irrelevant* only when the target language and extraction policy have established that removing it cannot change the retained observable behavior. That criterion may be syntactic, semantic, universe-specific, or proof-assistant-specific. Volume III uses a deliberately small marked fragment: terms annotated **ghost** may be erased only when the checker verifies that no retained computation depends on their value.

Proposition 10.1 (Toy erasure preservation). *For the marked erasure fragment defined in the laboratory, if a term passes the relevance checker and evaluates to retained output v , then its erased form evaluates to the same retained output on the retained fixture semantics.*

Proof. Induct on the checked term. Ordinary constructors are preserved recursively. At a **ghost** binder, the relevance checker has established that the bound value occurs only in erasable positions, so deleting the binder and its erasable uses leaves every retained constructor and branch guard unchanged. The result is deliberately architecture-specific; it is not global proof irrelevance. $\square$

Two inhabitants can compute differently

An inhabitant of $A+B$ is either $\text{inl}(a)$ or $\text{inr}(b)$. A case consumer may return different results for the two tags. Quotienting these proofs merely because they inhabit the same proposition would erase a branch choice that the program observes. Proof irrelevance is therefore incompatible with this relevance-sensitive interpretation unless the observable language is changed.

RELEVANCE

 $\text{inl}(a)$
 $\text{inr}(b)$

same sum type, potentially different behavior

Figure 10.1: **Plate 28. Relevance: two inhabitants, two behaviors.** Same-typed evidence can drive different branches in a relevance-sensitive context. Scope: no global proof-irrelevance assumption.

ERASURE

proof-bearing term $\xrightarrow{\text{checked erasure}}$ retained program

Figure 10.2: **Plate 29. What erasure may remove.** Only structure proved irrelevant under the stated policy may disappear while retained behavior is preserved. Scope: toy marked fragment only.

PROOF IRRELEVANCE IS ADDITIONAL

 $\text{propositions-as-types}$
 $\text{does not imply all inhabitants equal}$

Figure 10.3: **Plate 30. Proof irrelevance is an additional principle.** Curry–Howard alone does not quotient inhabitants. Scope: systems may separately validate irrelevance in selected proposition universes.

Do not confuse

Some type theories deliberately validate proof irrelevance for selected proposition universes or proof fields. That design is coherent. The error would be to infer it automatically from Curry–Howard or to apply an erasure theorem outside the fragment for which its dependency conditions were proved.

Computational Laboratory

Run `python labs/lab10_erasure.py`. It compares relevant and erasable fixtures, rejects a hostile case where a supposedly erased value controls retained output, and records both traces.

Exercises

- 10.1. Checkpoint.** Define proof relevance operationally in one sentence.
- 10.2. Checkpoint.** Why does a sum proof expose relevance?
- 10.3. Checkpoint.** Is proof irrelevance assumed by CH-0?
- 10.4. Core.** Give two same-typed terms whose observable results differ in a relevance-sensitive context.
- 10.5. Core.** Mark an argument that can safely be erased under the toy policy.
- 10.6. Core.** Give an argument that must not be erased because it controls a branch.
- 10.7. Synthesis.** Distinguish semantic proof irrelevance from compiler dead-code elimination.
- 10.8. Synthesis.** Explain how erasure changes the meaning of “the proof is the program.”
- 10.9. Proof Workshop.** Prove the toy erasure proposition for an ordinary application constructor.
- 10.10. Proof Workshop.** Explain the dependency invariant required at a ghost binder.
- 10.11. Design Clinic.** Specify a hostile fixture that catches an unsound erasure pass.
- 10.12. Challenge.** Describe a setting in which proof irrelevance is desirable and state what computational information is intentionally surrendered.

Chapter 11

Extraction

If a constructive proof already contains an executable term, why speak of extraction at all? Because practical proof artifacts usually contain structure that a runtime does not need: type annotations, proof-only arguments, certificates, logical wrappers, and sometimes opaque or axiomatic ingredients. Extraction is a controlled transformation from a proof-bearing artifact to a retained program.

This volume makes only example-level extraction claims. The selected examples start from proof terms in calculi whose computational content is explicit, apply a declared erasure/translation, evaluate the retained program, and replay the concrete specification against the output.

Proposition 11.1 (Witness preservation in the selected existential extractor). *For the finite Σ -style examples admitted by `lab11_extractor.py`, if the checker accepts a package $\langle a, p \rangle : \Sigma(x : A).B(x)$ and the extractor returns witness a , then replay of the registered decidable predicate for B accepts that returned witness.*

Proof. The laboratory’s accepted package format contains the witness and a certificate produced by the same decidable predicate used for replay. Acceptance first checks the witness domain and certificate. Extraction projects the witness without changing it. Replay recomputes the predicate on that exact value. Therefore accepted fixtures satisfy the registered predicate after extraction. The proposition is about this concrete architecture, not arbitrary proof-assistant extraction. $\square$

A proof that returns the witness

Suppose the finite goal is $\exists n \in \{0, 1, 2, 3, 4\}. n^2 = 4$. A constructive proof can carry $n = 2$ together with checked evidence for $2^2 = 4$. The extractor keeps 2 and discards the certificate after checking. Runtime output therefore preserves the computationally relevant witness.

Extraction becomes more delicate when the proof uses classical axioms, opaque constants, effects, quotienting, or principles whose computational interpretation is external. A production system may translate such ingredients, reject extraction, retain runtime support, or erase them under a justified irrelevance theorem. The correct statement is system-specific.

Do not confuse

The examples do not establish a general extraction-correctness theorem for Coq, Lean, Agda, or any other industrial proof assistant. Those systems have their own calculi, kernels, erasure rules, code generators, and trust boundaries.

Computational Laboratory

Run `python labs/lab11_extractor.py`. It checks finite witness/certificate packages, extracts accepted witnesses, replays the registered predicates, and rejects corrupted certificates.

Exercises

- 11.1. Checkpoint.** Distinguish evaluation from extraction.
- 11.2. Checkpoint.** What information is retained from the selected Σ example?
- 11.3. Checkpoint.** Why is replay performed after extraction?
- 11.4. Core.** Give a finite existential proposition and a witness-producing proof object.
- 11.5. Core.** Identify which fields of that object a runtime needs.
- 11.6. Core.** Construct a corrupted certificate that the extractor must reject before output.
- 11.7. Synthesis.** Explain how proof irrelevance and extraction can interact without being the same notion.
- 11.8. Synthesis.** List three ingredients that can complicate executable extraction.
- 11.9. Proof Workshop.** Prove witness preservation for the finite package format.
- 11.10. Proof Workshop.** State an end-to-end extraction theorem stronger than the one proved here and list its extra obligations.
- 11.11. Design Clinic.** Separate checker, extractor, and replay responsibilities in an API.
- 11.12. Challenge.** Explain why a successful extracted executable is evidence about a pipeline but not by itself a proof that the source theorem was formalized correctly.

EXTRACTION PIPELINE

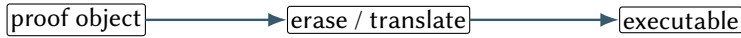


Figure 11.1: **Plate 31. Extraction pipeline.** A checked proof-bearing artifact is transformed into retained executable content. Scope: exact only for the declared extraction examples.

WITNESS SURVIVES



Figure 11.2: **Plate 32. Witness-preserving extraction.** Selected existential constructions retain the witness while proof certificates are checked and discarded. Scope: finite registered predicates only.

EXTRACTION PRESSURE POINTS

axioms
classical principles
opaque constants
effects

Figure 11.3: **Plate 33. Extraction pressure points.** Several ingredients require system-specific treatment before executable meaning is claimed. Scope: illustrative taxonomy, not a universal failure theorem.

Chapter 12

Uniformity Has Teeth

A polymorphic type can constrain behavior even before we know the implementation. Relational parametricity makes that fact precise: a polymorphic term must act uniformly with respect to relations between its type instantiations, subject to the hypotheses of the parametric model.

Volume III treats this as a preview in the restricted layer CHF-1. We do not build a complete System F kernel. We use a small polymorphic syntax for selected examples, cite the relational abstraction theorem, and compute finite relational instances.

Imported theorem: relational parametricity

Reynolds’s abstraction theorem interprets types relationally and shows, for the corresponding polymorphic calculus and semantics, that well-typed terms preserve the relations assigned to their type variables. The exact theorem is imported; finite checks below illustrate consequences but do not establish the theorem.

Consider a closed pure term of polymorphic type

$$\forall\alpha. \alpha \rightarrow \alpha.$$

Under the standard parametricity hypotheses, its behavior is severely constrained: it cannot manufacture an arbitrary element of an unknown type, inspect a type-specific representation, or choose a distinguished value unavailable from its argument. The familiar consequence is identity-like behavior, modulo the exact calculus and observational equality.

Proposition 12.1 (Finite relational fixture property). *For every finite relation enumerated by `lab12_parametricity.py`, each registered exemplar marked `parametric` maps related inputs to related outputs.*

Proof. The laboratory exhaustively enumerates the finite carrier pairs and relation pairs in its declared fixture universe. It evaluates each registered exemplar on both components of every related input pair and tests membership of the resulting pair in the target relation.

PARAMETRICITY AS UNIFORMITY

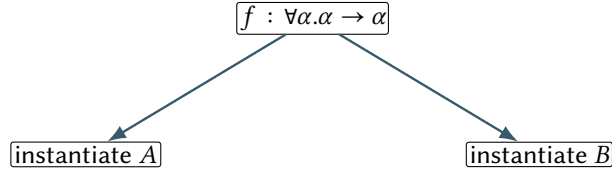


Figure 12.1: **Plate 34. Parametricity as uniformity.** A polymorphic term must act coherently across type instantiations under relational-parametricity hypotheses. Scope: CHF-1 preview.

RELATIONS TRAVEL THROUGH TERMS



Figure 12.2: **Plate 35. Relations travel through a polymorphic term.** Relational interpretation constrains paired outputs whenever paired inputs are related. Scope: imported theorem; finite lab checks only selected instances.

The proposition therefore follows from exhaustive finite enumeration over that fixture universe only. $\square$

Do not confuse

A finite relation checker is not a proof of relational parametricity for System F. Effects, runtime type analysis, nontermination, unsafe casts, or a richer observational theory can invalidate naive free-theorem reasoning unless the theorem's hypotheses are restored.

Computational Laboratory

Run `python labs/lab12_parametricity.py`. The program exhaustively checks the retained finite relations and includes a deliberately nonuniform hostile function that must fail at least one relation.

Exercises

- 12.1. Checkpoint.** What does parametricity add beyond ordinary typing?
- 12.2. Checkpoint.** Why can $\forall\alpha.\alpha \rightarrow \alpha$ not normally manufacture a new α ?
- 12.3. Checkpoint.** What is the evidentiary status of the finite checker?
- 12.4. Core.** Test identity against a relation of your choice.
- 12.5. Core.** Construct a nonuniform finite function that violates a relation.
- 12.6. Core.** Explain why runtime type inspection changes the argument.
- 12.7. Synthesis.** Relate parametric uniformity to abstraction barriers.
- 12.8. Synthesis.** State two hypotheses needed before using a free theorem.
- 12.9. Proof Workshop.** Prove the finite fixture property from exhaustive enumeration.
- 12.10. Proof Workshop.** State the relational interpretation of an arrow type informally but precisely.
- 12.11. Design Clinic.** Design a relation-generator that avoids vacuous all-empty test suites.
- 12.12. Challenge.** Explain why parametricity is a deeper constraint than checking a handful of type instantiations independently.

FREE-THEOREM CONSTRAINT

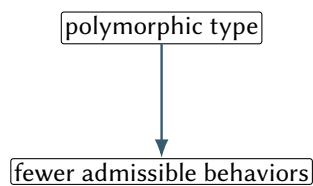


Figure 12.3: **Plate 36. Free-theorem constraint surface.** Parametric type structure can rule out implementations unavailable from ordinary monomorphic typing alone. Scope: purity and parametricity hypotheses are essential.

Chapter 13

Proof Assistants Have a Trust Boundary

“The proof is the program” can obscure the machinery between a human command and a kernel judgment. A modern proof assistant may include a parser, elaborator, unifier, tactic engine, simplifier, compiler, external solver, and a small trusted checker. The proof object matters because it lets a smaller component recheck what larger untrusted components propose.

The toy architecture in this chapter has two components. An untrusted elaborator turns abbreviated surface declarations into fully explicit CH-0 terms. A separate kernel receives only the explicit term, context, and claimed type. It parses independently and runs the small typing relation from first principles.

Proposition 13.1 (Kernel rechecking boundary). *In the toy architecture, acceptance by the independent kernel depends only on the explicit object supplied to the kernel and the kernel’s implemented rules; it does not depend on the elaborator’s claim that the object is valid.*

Proof. The kernel does not consume an “accepted” flag from the elaborator. It reparses the explicit certificate and recomputes the typing judgment. Hostile fixtures alter either a binder type, an application argument, or a claimed result type while preserving the elaborator-shaped wrapper. These fixtures are rejected. Thus elaborator authority is not part of the kernel acceptance rule. $\square$

This proposition is architectural, not metaphysical. Trust has not disappeared. It has been concentrated into the kernel implementation, parser, runtime, foundational calculus, and the correctness of the statement submitted for checking. A tiny kernel can faithfully certify the wrong formalization of an informal theorem.

A plausible but invalid certificate

Suppose an elaborator claims that $(\lambda x:P.x)$ has type $Q \rightarrow Q$. The syntax is well formed and the term is typable, but the claimed type is wrong unless P and Q are definitionally the same atomic type. Independent rechecking derives $P \rightarrow P$ and rejects the mismatched certificate.

Do not confuse

Kernel acceptance establishes validity according to the encoded rules and statement. It does not establish that the informal theorem was translated correctly, that the kernel has no implementation bug, or that the foundational assumptions are philosophically mandatory.

Computational Laboratory

Run `python labs/lab13_kernel_boundary.py`. Positive certificates must pass; hostile mutated certificates must be rejected by the independent checker. The lab measures this toy trust boundary only.

Exercises

- 13.1. **Checkpoint.** Distinguish elaboration from kernel checking.
- 13.2. **Checkpoint.** What input does the toy kernel trust from the elaborator?
- 13.3. **Checkpoint.** What remains in the trusted computing base?
- 13.4. **Core.** Mutate a valid certificate so only the claimed result type is wrong.
- 13.5. **Core.** Explain why reparsing independently matters.
- 13.6. **Core.** Give an example of a correct formal proof of the wrong informal statement.
- 13.7. **Synthesis.** Compare proof objects with proof-carrying certificates in other systems.
- 13.8. **Synthesis.** Explain why a smaller kernel can improve auditability without creating absolute certainty.
- 13.9. **Proof Workshop.** Prove the stated kernel-boundary proposition from the interface definition.
- 13.10. **Proof Workshop.** Classify parser correctness as a separate trust obligation.
- 13.11. **Design Clinic.** Minimize a checker interface while retaining explicit diagnostics.
- 13.12. **Challenge.** Draw the full trust cone from informal theorem to hardware and identify which layers formal proof does and does not eliminate.

TACTIC / ELABORATOR / KERNEL

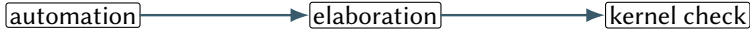


Figure 13.1: **Plate 37. Tactic, elaborator, kernel.** Convenience layers propose explicit objects; the kernel independently checks the final certificate. Scope: toy architecture abstracting real systems.

THE TRUST CONE



Figure 13.2: **Plate 38. The kernel trust cone.** Independent rechecking narrows authority to the checker, parser, calculus, runtime, and statement formalization. Scope: trust is concentrated, not eliminated.

HOSTILE CERTIFICATE REJECTION



Figure 13.3: **Plate 39. Hostile certificate rejection.** A malformed elaborator-shaped object fails independent kernel rechecking. Scope: demonstrates the laboratory boundary only.

Chapter 14

Where Proof / Program Stops Being Enough

The strongest results in this volume concern constructions that can be represented as terms and simplified toward values or normal forms. But many correct computations are not supposed to finish. A server, channel endpoint, operating system, or agent may remain live indefinitely because continued interaction is its purpose.

A closed function type $A \rightarrow B$ says: given an A , construct a B . It does not by itself say who sends first, which messages may follow which, whether two peers have complementary obligations, whether a resource may be used exactly once, or whether a well-typed conversation can deadlock.

The teaching laboratory contrasts two objects:

1. a closed term that evaluates to a final value;
2. a two-party state machine whose successful execution is an indefinitely extensible trace of legal message transitions.

For the second object, “normal form” is not the central success criterion. A permanently quiescent endpoint may be broken; a nonterminating endpoint may be correct.

Proposition 14.1 (Closed normalization does not imply protocol correctness). *There exists a finite-state two-party protocol representation whose local message handlers are terminating functions while the protocol as a whole can violate a declared peer-ordering obligation.*

Proof. Take two terminating handlers for messages `hello` and `ack`. Define the protocol obligation that the client sends `hello` before receiving `ack`, while a malformed peer emits `ack` in the initial state. Each handler terminates on its input, yet the trace violates the protocol transition relation at its first step. Thus termination of the component functions does not imply trace conformance. $\square$

What is missing is a type for interaction itself. We need complementary endpoint obligations, channel state, controlled resource use, branching/selection, recursion, and metatheorems that distinguish communication safety from progress and deadlock freedom. Those are not decorative extensions of the closed-term story. They change the primary computational object.

Governing Principle

Volume III has reached its boundary when the question changes from “what value does this construction produce?” to “what obligations must two evolving computations maintain toward each other?”

Do not confuse

The protocol laboratory does not establish session fidelity, deadlock freedom, distributed-system correctness, fairness, or liveness. It demonstrates why those become new theorem obligations.

Computational Laboratory

Run `python labs/lab14_protocol_threshold.py`. It compares a terminating closed evaluator with legal and hostile protocol traces. The hostile trace shows that local termination and typing are insufficient for interaction correctness.

14.1 What survives the volume

The evidence now supports a stratified conclusion. In pure constructive fragments, proof construction and program construction can share exact rule/term structure, and principal proof normalization can correspond to program reduction. Dependent quantifiers extend this reading through witness-sensitive construction. Sequent cut elimination is closely related through translation, not vocabulary. Classical reasoning may require translation or control. Erasure and extraction require explicit relevance policies. Parametricity adds semantic uniformity hypotheses. Proof assistants insert a trust architecture between surface proof and kernel acceptance. Open interaction finally exceeds the closed-value model.

The residual thesis is stronger for being narrower: type theory provides a disciplined grammar for computational possibility, but the grammar must grow when the computational object changes.

Exercises

14.1. Checkpoint. Why can correct interaction be nonterminating?

CLOSED TERM / OPEN CONVERSATION



Figure 14.1: **Plate 40. Closed term versus open conversation.** Termination is natural for closed evaluation but can be the wrong success criterion for an interactive service. Scope: state-machine preview, not session typing.

THE MISSING DUAL



Figure 14.2: **Plate 41. The missing dual.** Open interaction needs complementary endpoint obligations rather than one closed output type. Scope: previews session-type duality without formalizing it.

THRESHOLD TO PROTOCOL



channel · duality · protocol state · progress · deadlock

Figure 14.3: **Plate 42. Threshold to PROTOCOL.** The series moves from typed construction to typed interaction because closed normalization cannot express bilateral protocol obligations. Scope: the ten-volume unification remains a research thesis.

- 14.2. **Checkpoint.** Name one fact not expressed by a plain $A \rightarrow B$ type.
- 14.3. **Checkpoint.** Distinguish local handler termination from trace conformance.
- 14.4. **Core.** Construct a two-state request/response protocol.
- 14.5. **Core.** Give a hostile trace that violates its ordering rule.
- 14.6. **Core.** State separate safety and liveness questions for the protocol.
- 14.7. **Synthesis.** Explain why session fidelity is not the same as distributed-system correctness.
- 14.8. **Synthesis.** Summarize the strongest form of Curry–Howard correspondence that survives Chapters 1–14.
- 14.9. **Proof Workshop.** Prove the closed-normalization/protocol-correctness separation proposition with an explicit trace.
- 14.10. **Proof Workshop.** Identify the additional hypotheses needed to turn local progress into a global communication-progress theorem.
- 14.11. **Design Clinic.** Specify the state and transition data a typed protocol language should expose.
- 14.12. **Challenge.** Draft the minimum formal vocabulary Volume IV must add before duality and deadlock claims can be meaningful.

Formal reference and notation guide

Γ, Δ	typing/proof contexts.
$\Gamma \vdash t : A$	term t has type A under context Γ .
$\Gamma \vdash A$	derivability with the selected proof term suppressed.
$\llbracket D \rrbracket$	proof term assigned to derivation D .
$t \longrightarrow t'$	one operational reduction step in the stated calculus/strategy.
$t \longrightarrow^* t'$	zero or more operational steps.
$t \equiv u$	calculus-scoped definitional/equational equality; not one-step reduction.
$A \times B, A + B$	product/conjunction and sum/disjunction type formers in CH-0.
1, 0	unit/truth and empty/falsity types.
$\Pi(x : A).B(x), \Sigma(x : A).B(x)$	dependent product/universal and dependent sum/existential constructions in CHD-1.
$\Gamma \Rightarrow A$	single-succedent sequent in LJ-0.
$w \Vdash A$	Kripke-world forcing relation used in Chapter 8.
R	local answer type in Chapter 9 or local relation in Chapter 12; the chapter fixes the intended meaning.

Course and self-study roadmap

Chapters 1–5 form a constructive core suitable for a first proof/program module. Chapters 6–9 form the logic-boundary module: dependent quantification, sequent cut, intuitionistic/-classical separation, and control. Chapters 10–13 form the implementation-and-evidence module: relevance, extraction, parametricity, and kernel trust. Chapter 14 is the transition seminar into typed interaction. Each teaching chapter has twelve exercises and one executable laboratory; the solutions companion contains a keyed solution or rubric for every exercise.

Illustration and production note

The volume contains 42 canonical TikZ plates. Each plate carries a single pedagogical burden and an explicit scope or analogy limit. Computed claims represented in plates remain bound to the corresponding laboratory/evidence records. Visual closure requires inspection at manuscript and folio scale; source conformance alone does not satisfy that gate.

Selected bibliography

Bibliography

- [1] Alonzo Church. “A Formulation of the Simple Theory of Types.” *Journal of Symbolic Logic* 5(2), 1940, 56–68.
- [2] Gerhard Gentzen. “Untersuchungen über das logische Schließen I.” *Mathematische Zeitschrift* 39, 1935, 176–210.
- [3] Dag Prawitz. *Natural Deduction: A Proof-Theoretical Study*. Almqvist & Wiksell, 1965.
- [4] William A. Howard. “The Formulae-as-Types Notion of Construction.” In J. P. Seldin and J. R. Hindley, eds., *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, Academic Press, 1980, 479–490.
- [5] Jean-Yves Girard. *Interprétation fonctionnelle et élimination des coupures de l’arithmétique d’ordre supérieur*. Doctoral thesis, Université Paris VII, 1972.
- [6] John C. Reynolds. “Types, Abstraction and Parametric Polymorphism.” In *Information Processing* 83, 513–523.
- [7] Timothy G. Griffin. “A Formulae-as-Types Notion of Control.” *Proceedings of POPL*, 1990, 47–58.
- [8] Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, 2002.
- [9] Morten Heine Sørensen and Paweł Urzyczyn. *Lectures on the Curry–Howard Isomorphism*. Elsevier, 2006.
- [10] Robert Harper. *Practical Foundations for Programming Languages*, 2nd ed. Cambridge University Press, 2016.

Transition: from construction to interaction

The volume began with an exact question about constructions: when a proof is assembled from introduction and elimination rules, when does its structure also define a program? The answer survives substantial pressure, but not unchanged. The final boundary is structural. An indefinitely open conversation is not merely a closed term that has failed to normalize. It has peer-relative obligations, protocol state, branching, resource use, and liveness questions. Volume IV therefore replaces the closed input–output object with typed interaction.

Index

- beta reduction, 7
- canonical forms, 22
- case analysis, 19
- certificate replay, 48
- classical logic, 36
- conjunction, 15
- continuation-passing style, 40
- control operator, 40
- Curry–Howard correspondence, 1
- cut elimination, 32
- cut rule, 32
- deadlock, 57
- dependent product, 27
- dependent sum, 27
- disjunction, 19
- double negation, 40
- duality, 57
- elaborator, 54
- erasure, 44
- excluded middle, 36
- existential quantification, 27
- interaction, 57
- intuitionistic logic, 36
- kernel, 54
- Kripke semantics, 36
- liveness, 57
- natural deduction, 1
- normalization, 7
- parametricity, 50
- polymorphism, 50
- preservation, 22
- product type, 15
- program extraction, 48
- progress, 22
- proof assistant, 54
- proof irrelevance, 44
- proof relevance, 44
- proof term, 1
- protocol, 57
- relational interpretation, 50
- sequent calculus, 32
- strong normalization, 22
- substitution, 7
- sum type, 19
- trusted computing base, 54
- universal quantification, 27
- witness, 48